

Program for Problem Solving (CS101)

Mayur V. Shrirame

(mayurshrirame@gmail.com)



Birla Institute of Technology, Mesra
Ranchi

My Rules, You must follow:-

- No bargaining of marks.
- No proxy attendance in any case.
- Late Latif's, stay outside.
- No postponing of any quiz or class test.
- No Parents-Teacher meet for marks updation.
- You can play games, text someone, gossips, etc but

After my lecture is over

Module I

[9L]

Introduction to Programming:

Introduction to components of a computer system (disks, memory, processor, where a program is stored and executed, operating system, compilers etc.)

Problem Solving: Steps to solve logical and numerical problems.

Representation of Algorithm: Flowchart/Pseudo code with examples. From algorithms to programs; source code, variables (with data types) variables and memory locations, Syntax and Logical Errors in compilation, object and executable code

Module II

[9L]

Arithmetic expressions and precedence, Conditional Branching and Loops, Writing and evaluation of conditionals, Iterations, Loops.

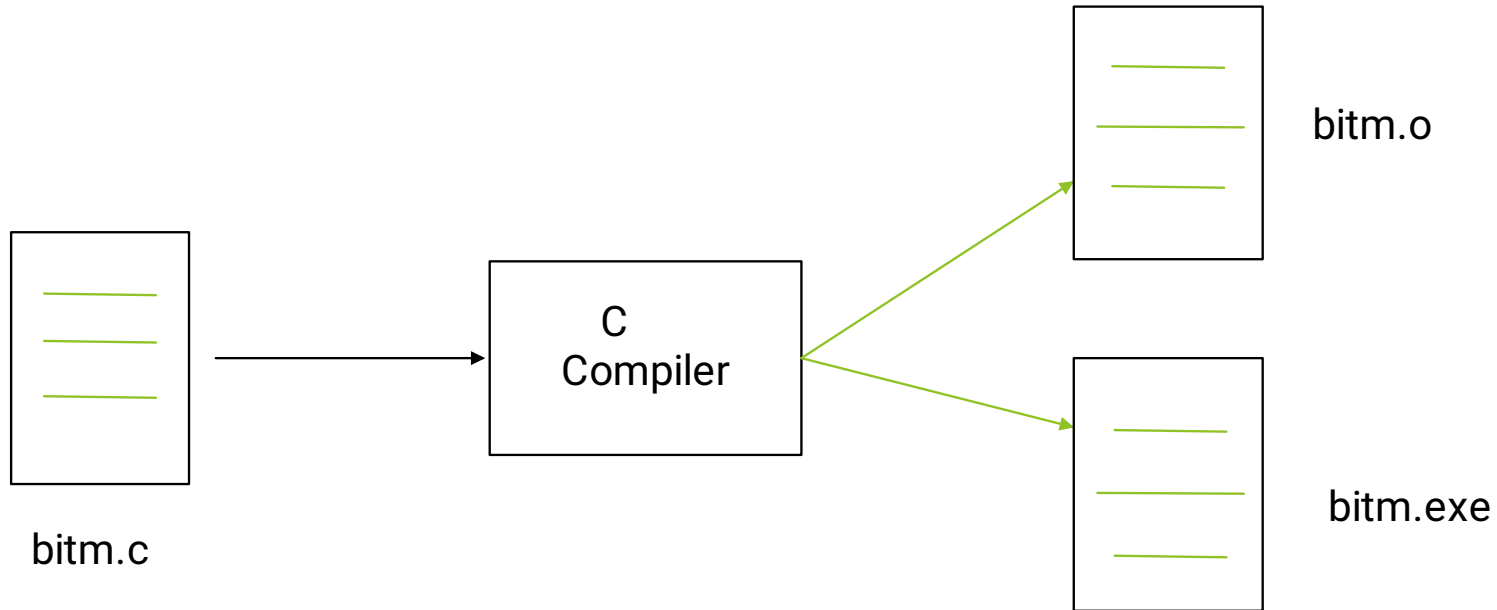
Module III

[9L]

Array, Character array, strings. Case studies to discuss the various Problems related to Basic science (Matrix addition, Matrix-matrix multiplication, Roots of an equation etc.), Sorting, Searching.

Behind the Scenes

- What happens when a C program is compiled ?



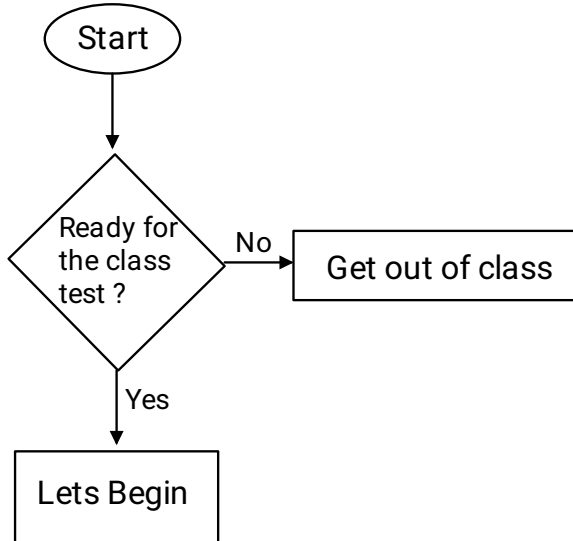
How a problem is solved ?

Step 1:- Draw a flowchart.

Step 2:- Write an algorithm according to flowchart.

Step 3:- Choose any programming language to formulate the algorithm into code.

Flowchart



Algorithm

1. Start
2. Ready for the class test ?
3. If Yes, goto 5
4. If No, goto 6
5. Lets Begin
6. Get out of class
7. Exit

Code

```
#include<stdio.h>
void main()
{
    class_test=0;
    if(class_test==1)
        printf("Lets Begin!");
    else
        printf("Get out of class");
}
```

Let's dive into the world of



C

A sea (C Language) holds many creatures (concepts) in it. They are :-

- Living Creatures (Utmost Important) -----→ 90 %
- Non-Living Creatures (You can skip it) -----→ 10 %

1. Living Creatures

- a. Conditional statements
- b. Loops
- c. Functions
- d. Arrays
- e. Strings
- f. Structures
- g. Pointers
- h. File Handling

2. Non Living Creatures

- a. Graphics Programming

My First C Program

- What is your roll number ?

```
#include <stdio.h>
void main()
{
    int roll_no;
    printf("\n What is your roll number: ");
    scanf(" %d", &roll_no);
    printf("\n Your roll number is: %d", roll_no);
}
```

Variables, Constants & Keywords

- A variable is an entity to store some value.
 - Example:- roll_no, simpleInterest, Average, etc
- A constant is any value.
 - Example: 1,-2, 'm', "mayur", etc
- A keyword is a predefined words in C Language. There are total 32 predefined keywords in C.
 - Example:- if, else, while, include, etc

Tip:- Never use predefined keywords as variable name.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while



Rules for Variable Creation

Rules for Constructing Variable Names

- (a) A variable name is any combination of 1 to 31 alphabets, digits or underscores. Some compilers allow variable names whose length could be up to 247 characters. Still, it would be safer to stick to the rule of 31 characters. Do not create unnecessarily long variable names as it adds to your typing effort.
- (b) The first character in the variable name must be an alphabet or underscore.
- (c) No commas or blanks are allowed within a variable name.
- (d) No special symbol other than an underscore (as in **gross_sal**) can be used in a variable name.

- 12a, _bitm, bit12, bitMesra, bitm_, bit m, \$bitmesra



Operators in C

	Operator	Type
Unary operator →	+, -, ~	Unary operator
Binary operator {	+, -, *, /, %	Arithmetic operator
	<, <=, >, >=, ==, !=	Relational operator
	&&, , !	Logical operator
	&, , <<, >>, ~, ^	Bitwise operator
	=, +=, -=, *=, /=, %=	Assignment operator
Ternary operator →	?:	Ternary or conditional operator

Operators allow us to perform different kinds of operations on operands. In C, operators can be categorized in following categories:

- **Arithmetic Operators** (+, -, *, /, %, ++, --)
- **Relational Operators** (==, !=, >, <, >= & <=) **Logical Operators** (&&, || and !)
- **Bitwise Operators** (&, |, ^, ~, >> and <<)
- **Assignment Operators** (=, +=, -=, *=, etc)

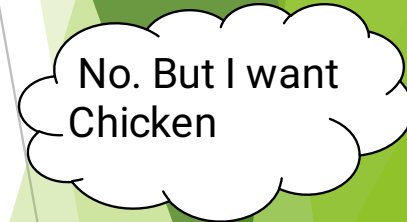
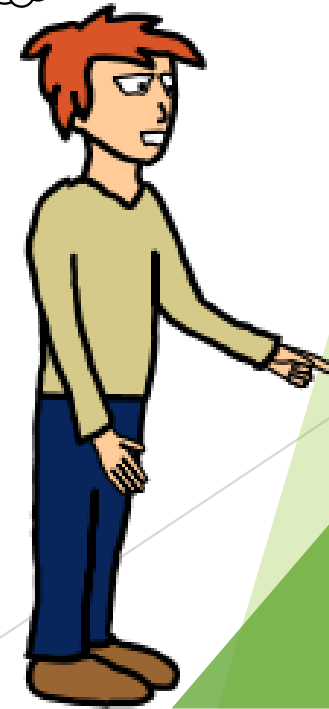
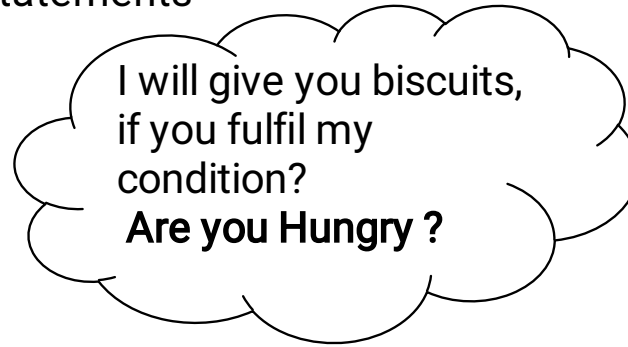
Precedence of Operators

Description	Operator	Associativity
Function expression	()	Left to Right
Array Expression	[]	Left to Right
Structure operator	->	Left to Right
Structure operator	.	Left to Right
Unary minus	-	Right to left
Increment/Decrement	++ --	Right to Left
One's compliment	~	Right to left
Negation	!	Right to Left
Address of	&	Right to left
Value of address	*	Right to left
Type cast	(type)	Right to left
Size in bytes	sizeof	Right to left
Multiplication	*	Left to right
Division	/	Left to right
Modulus	%	Left to right
Addition	+	Left to right
Subtraction	-	Left to right
Left shift	<<	Left to right
Right shift	>>	Left to right
Less than	<	Left to right
Less than or equal to	<=	Left to right
Greater than	>	Left to right
Greater than or equal to	>=	Left to right
Equal to	==	Left to right
Not equal to	!=	Left to right

Description	Operator	Associativity
Bitwise AND	&	Left to right
Bitwise exclusive OR	^	Left to right
Bitwise inclusive OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	? :	Right to left
Assignment	=	Right to left
	*= /= %=	Right to left
	+= -= &=	Right to left
	<<= >>=	Right to left
	<<<= >>>=	Right to left
Comma	,	Right to left

Decision Control Statements

1. `if(expression) // normal if-else`
 Statement 1;
 else
 Statement 2
2. `if(expression) // else if ladder`
 {
 Statement 1;
 Statement 2;
 }
 else if(expression)
 {
 Statement 3;
 Statement 4;
 }
 else if(expression)
 {
 Statement 5;
 }
 else
 {
 Statement 6;
 }



3. if(expression) // nested if else

```
{  
  if(expression)  
  {  
    Statement 1;  
    Statement 2;  
  }  
  else  
  {  
    Statement 3;  
    Statement 4;  
  }  
}  
else  
{  
  Statement 5;  
  Statement 6;  
}
```

Logical Operators

1. Logical AND (&&)
When both the conditions are true.
 2. Logical OR (||)
When only one condition is true.
 3. Logical NOT (!)
-
- Take year as input from user and find whether it is leap year or not.

Switch-case Statement

```
switch(integer expression)
{
    case 1: something;
        break;
    case 2: something;
        break;
    case 3: something;
        break;
    default: something;
}
```

- Any order you can have in case.
- In case, you used char those values are converted to ASCII.
- Same case can be executed (case 'a':case 'A').
- No variables in case.

GOTO Statement

- goto label_name;
- Goto statement is used to transfer the control to another statement.

```
#include<stdio.h>
void main()
{
    int i;
    scanf("%d", &i);
    if(i<=100)
        goto Label1;
    else
    {
        printf("Never Use goto statement");
        printf("This is the last time we are using goto statement");
    }
    Label1:
    printf("Don't use goto, its an order");
}
```

Loop Control Statements

1. for loop
2. while loop
3. do-while loop

- For loop (Entry controlled Loop)

```
int i;
```

```
for( i=0; i<10; i++)
```

```
{
```

```
    printf("\n Welcome");
```

```
}
```

- Initialization (i=0)
- Condition (i<10)
- Increment/Decrement (i++/i--)

- Prints Welcome 10 times.

- While Loop (Entry controlled Loop) → 1) initialization

```
int i;
```

```
i=0;
```

```
while(i<10)
```

```
{
```

```
printf("\n The value of i is: %d", i);
```

```
i++;
```

```
}
```

2) condition

3) increment/decrement

- Do-while Loop (Exit Controlled Loop)

```
int i;
```

```
i=0;
```

```
do
```

```
{
```

```
    printf("\n Hello");
```

```
    printf("\n BIT");
```

```
    i++;
```

```
}
```

```
while(i<10);
```

1) Initialization

3) Increment/Decrement

2) Condition

Break & Continue Statements

- Break statement is used to come out of the loop without executing other statements.
- Continue statement is used to start the loop with next iteration.

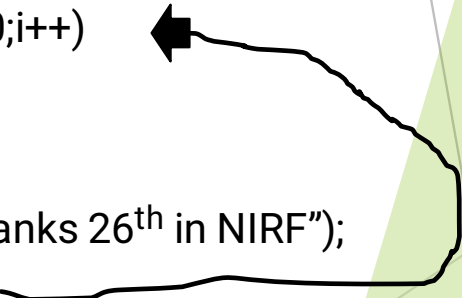
Break Statement

```
#include<stdio.h>
void main()
{ int i;
  for (i=0;i<10;i++)
  {
    if(i==7)
    {
      printf("\n Ranks 26th in NIRF");
      break;
    }
  }
  printf("\n BIT");
}
```



Continue Statement

```
#include<stdio.h>
void main()
{ int i;
  for (i=0;i<10;i++)
  {
    if(i>=7)
    {
      printf("\n Ranks 26th in NIRF");
      continue;
    }
    printf("\n BIT");
  }
}
```



How to solve pre increment/decrement and post increment/decrement problems ? (Compiler Dependent)

1

```
#include<stdio.h>
void main()
{
int a=10,i;
i = ++a + ++a + ++a;
printf("\n Value of a is: %d", a);
printf("\n Value of I is: %d", i);
}
```

2

```
#include<stdio.h>
void main()
{
int a=10,i;
i = a++ + a++ + a++;
printf("\n Value of a is: %d", a);
printf("\n Value of I is: %d", i);
}
```

3

```
#include<stdio.h>
void main()
{
int a=10,i;
i = --a + --a + --a;
printf("\n Value of a is: %d", a);
printf("\n Value of I is: %d", i);
}
```

4

```
#include<stdio.h>
void main()
{
int a=10,i;
i = a-- + a-- + a--;
printf("\n Value of a is: %d", a);
printf("\n Value of I is: %d", i);
}
```

Nested Loops

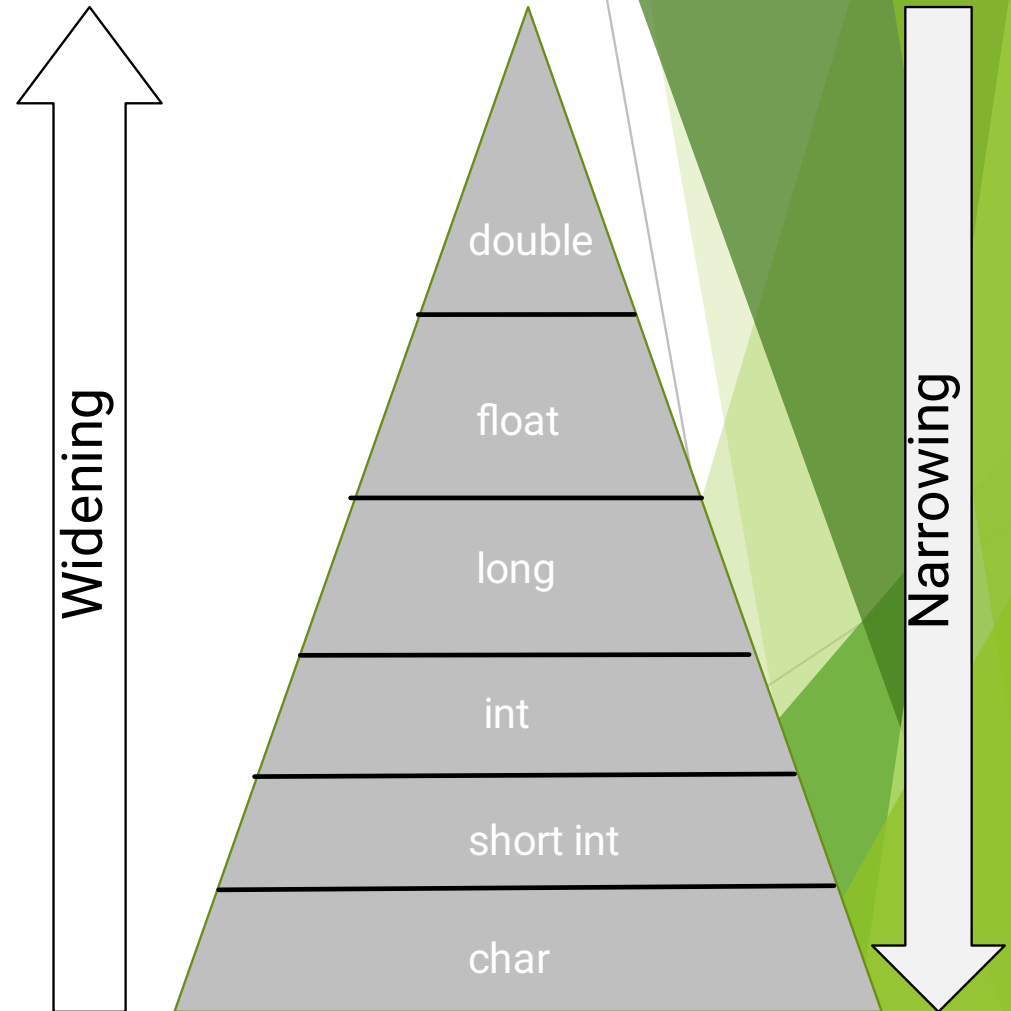
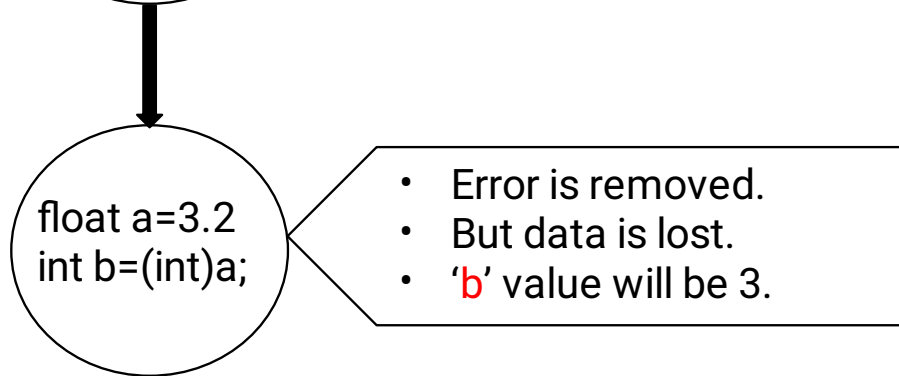
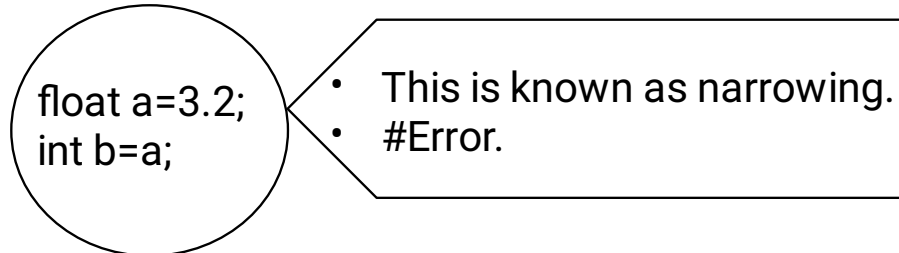
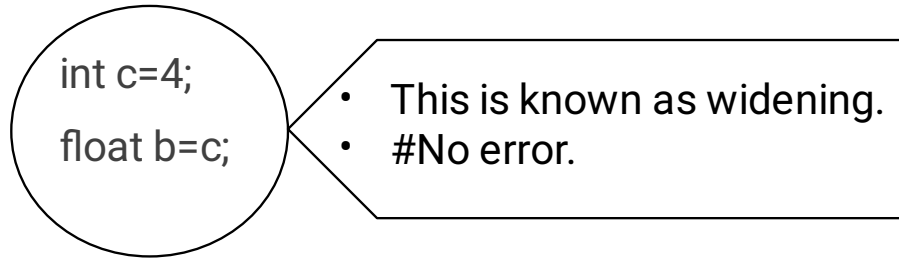
```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int rows;
    int i,j;
    char ch;
    while(1)
    { printf("\n Do you wish to continue(y/n): ");
      fflush(stdin);
      scanf("%c",&ch);
      switch(ch)
      {
      case 'y':
      case 'Y':
        printf("\n Enter number of rows: ");
        scanf("%d",&rows);
        for(i=0;i<rows;i++)
        {
            for(j=0;j<=i;j++)
            { printf("*"); }

            printf("\n");
        }
        break;
      case 'n':
      case 'N': exit(0);
        break;
      default: printf("\n Enter correct character.");
      }
    }
    return 0;
}
```

Questions on Loops/ switch

- Write a menu driven program for pizza delivery shop. A user will first choose base (bread) pizza and its cost will get added to the total amount. Next user can choose pan-baked, cheese burst or normal baked (amount will be added). Next user will be asked to choose toppings among few, olives, macaroni, corn, paneer, (amount will be added). Maximum of 3 toppings can be added. A user can delete any topping when he/she wants to. Finally display the total amount to be paid by the user.
- Write a C program to find the multiplication table of a number.
- Write a C program for banking system. User can choose either to deposit or withdraw money from the bank and correspondingly amount must be reflected in the balance.
- Write a C program to check whether entered number is Armstrong number.

TypeCasting



Arrays (Static Array)

1. Define Array.

Ans: * Array :

An Array is used to call a boy or a person who is at a distance far away from us who are visible to our naked eye.

Example: Array Rupesh.....

Meet me
later.

- An array is a derived datatype which is a collection of similar elements.
- An array of integer elements or an array of character elements.
- Syntax:-

datatype array-name[size];

- `int marks[5];` → Single Dimension Array

```
#include<stdio.h>
void main()
{
    int marks[5], i;

    marks[0]=12;
    marks[1]=23;
    marks[4]=45;
    marks[2]=34;
    marks[3]=56;

    for(i=0;i<5;i++)
    {
        printf("\n Value at %d position is: %d", i, marks[i]);
    }
}
```

marks[0]	12
marks[1]	23
marks[2]	34
marks[3]	56
marks[4]	45

- Write a simple program to initialize character array with BIT MESRA and display the output.

Initialization

- **At compile time**

- Syntax:-

`datatype array-name[size]={ list of values};`

- Example:-

```
#include<stdio.h>
void main()
{
    int marks[5]= {12,23,34,45,56};
    int i;
    for(i=0;i<5;i++)
    {
        printf("\n Value at %d position is: %d", i, marks[i]);
    }
}
```

- **At run time**

- Example:-

```
#include<stdio.h>
void main()
{
    float average[3];
    int i;
    scanf("%f%f%f", &average[0], &average[1],
    &average[2]);

    for(i=0;i<3;i++)
    {
        printf("\n Value at %d position is: %f", i,
        average[i]);
    }
}
```

Two Dimensional Arrays

- Imagine a case where you want to store marks of 5 subjects of each roll number.

rollNumber	Subject_1	Subject_2	Subject_3	Subject_4	Subject_5
1001	30	40	50	60	30
1002	40	50	60	70	10
1003	50	54	34	20	58

Sorry you
can't add
this
particular
row.

- C Language allows us to define such tables using multi-dimensional arrays. The above case is an example of two dimensional arrays.

- Syntax:-

`datatype array-name[row-size][column-size];`

- Example:-

```
int markList[3][5] = { {1001,30,40,50,60,30}, {1002,40,50,60,70,10}, {1003,50,54,34,20,58} };
```

```
int markList[][5] = { {1001,30,40,50,60,30}, {1002,40,50,60,70,10}, {1003,50,54,34,20,58} };
```

```
int markList[3][5] = { {1001,30}, {1002, 40,50} };
```

```
int markList[3][5] = {1001,1002,30,40,50,60,30,1002 };
```

Initialization is
done row by
row.

Character Array

```
#include <stdio.h>
void main()
{
    int i;
    char address[10];
    scanf("%s",address);
    printf("%s",address);
}
```

1

```
#include <stdio.h>
void main()
{
    int i;
    char address[10];
    gets(address);
    printf("%s",address);
}
```

2

- To include whitespace in character array, use gets() function.
- Assignment, comparison is not allowed on strings.
Ex:- name[80]="BitMesra";
name2[80]=name;
- To tackle this situation, assign or compare character by character.
- Arithmetic operations are allowed on characters.
Ex:- char ch='a';
int x;
x= ch +1;

String Handling Functions

- strcmp()
 1. Compares two strings and returns 1 if ASCII value of string1 is greater than string2, returns 0 if ASCII value of string1 and string2 is same, returns -1 if ASCII value of string1 is smaller than string2.
- strcat()
 1. Concats two strings one after other.
- strcpy()
 1. Copies content of string2 to string 1.
- strlen()
 1. Returns the length of string.

```
#include <stdio.h>
#include<string.h>
void main()
{
int i,cmp1,len1;
char string1[10],string2[10];
gets(string1);
gets(string2);
```

```
cmp1=strcmp(string1,string2);
strcat(string2,string1);
strcpy(string1,string2);
len1=strlen(string2);
printf("\n%d %d",cmp1,len1);
printf("\n%s\n%s",string1,string2);
}
```

Linear Search

Key element



Marks[8] =

12	23	34	45	56	67	78	89
----	----	----	----	----	----	----	----

- Linear search is a simple searching algorithm which traverses given array to find the key element in the array.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a[100],n,number,i,c=0;
    printf("Enter no. of elements: ");
    scanf("%d",&n);
    printf("\nEnter array elements: ");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

```
printf("\n Enter key element to be
searched: ");
scanf("%d",&number);
for(i=0;i<n;i++)
{
    if(a[i]==number)
    {
        c++;
        break;
    }
}
```

```
if(c>=1)
{
    printf("\n%d found in the array
%d times.",number,c);
}
else
{
    printf("\n %d not found in the
array.",number);
}
return 0;
}
```

Binary Search

- To perform binary search, crucial condition is that elements in the array are sorted.
- Three terms are introduced here, $L=0$, $H=n-1$, $MID=L+H/2$.
- At first iteration, searching starts from first/second half of the array.
- This continues till the key element is found.

Binary Search										
	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0	1	2	3	M=4	5	6	7	8	H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5	6	M=7	8	H=9
23 > 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5, M=5	H=6	7	8	9
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int a[100],n,number,i,l=0,mid=0,h=0,c=0;
    printf("Enter no. of elements: ");
    scanf("%d",&n);

    printf("\nEnter array elements: ");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("\n Enter key element to be searched: ");
    scanf("%d",&number);
    l=0;
    h=n-1;
```

```
while(l<=h)
{
    mid=(l+h)/2;
    if(a[mid]>number)
    {
        h=mid-1;
    }
    else if(a[mid]<number)
    {
        l=mid+1;
    }
    else{
        printf("%d found at mid",number);
        c=1;
        break;
    }
}
if(c==0)
    printf("\n Not found");
return 0;
}
```

Selection Sort

- Comparing first element in the array with rest of the other elements.
- If first is greater than other element, swap it.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int a[100],n,i,j,temp;
    printf("Enter no. of elements: ");
    scanf("%d",&n);

    printf("\nEnter array elements: ");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

```
for(i=0;i<n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(a[j]<a[i])
        {
            temp=a[j];
            a[j]=a[i];
            a[i]=temp;
        }
    }
}
```

```
for(i=0;i<n;i++)
{
    printf("%d\t",a[i]);
}
return 0;
}
```

Bubble Sort

- Bubble Sort repeatedly sorts two adjacent elements in the list.

```
#include <stdio.h>
#include <string.h>
void main()
{
    int a[100], n, i, j, temp;
    printf("\nEnter number of elements: ");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
}
```

```
for(i=0; i<n-1; i++)
{
    for(j=0; j<n-1; j++)
    {
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
        }
    }
}
```

```
printf("\n");
for(i=0; i<n; i++)
{
    printf("%d\t", a[i]);
}
```

Insertion Sort

- Insertion sort swaps i th element with previous $A[0 \text{ to } i-1]$ elements.

```
#include <stdio.h>
void main()
{
    int a[100],n,i,j,temp;
    printf("\nEnter number of elements: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

```
for(i=1;i<n;i++)
{
    temp=a[i];
    j=i-1;
    while(temp<a[j] && j>=0)
    {
        a[j+1]=a[j];
        j--;
    }
    a[j+1]=temp;
}
printf("\n");
for(i=0;i<n;i++)
{
    printf("%d\t",a[i]);
}
}
```

Storage Classes in C

Storage Class tells about the features of a variables. Features include scope, lifetime and default value of a variable.

- Automatic Variables

- a. Scope :- Within the function block where it is declared.
- b. Lifetime :- Till the end of function where it is declared.
- c. Default Value :- Garbage Value.

- Static Variables

- a. Scope :- Within the function block where it is declared.
- b. Lifetime :- Till the program stops its execution.
- c. Default Value:- Zero.

- Register Variables

- a. Scope :- Local to the function in which it is declared.
- b. Lifetime:- Till the end of function/method block, in which the variable is defined.
- c. Default Value:- Any random value i.e garbage value.

- External Variables

- a. Scope :- Global i.e everywhere in the program. These variables are not bound by any function, they are available everywhere.
- b. Lifetime :- Till the program doesn't finish its execution, you can access global variables.
- c. Default Value :- Zero.

auto variable

```
#include<stdio.h>

void main()
{
    int detail;
    // or
    auto int details;    //Both are same
}
```

static variable

```
#include<stdio.h>

void test();    //function declaration (discussed in next topic)

int main()
{
    test();
    test();
    test();
}

void test()
{
    static int x = 0;    //a static variable
    x = x + 1;
    printf("%d\n",x);
}
```

file1.c

```
#include<stdio.h>
int a = 7; //global variable
void fun()
{
    a++;
    printf("%d", a);
    .....
    .....
}
```

file2.c

```
#include "file1.c";
main()
{
    extern int a;
    fun();
}
```

extern variable

```
#include<stdio.h>

int number;    // global variable

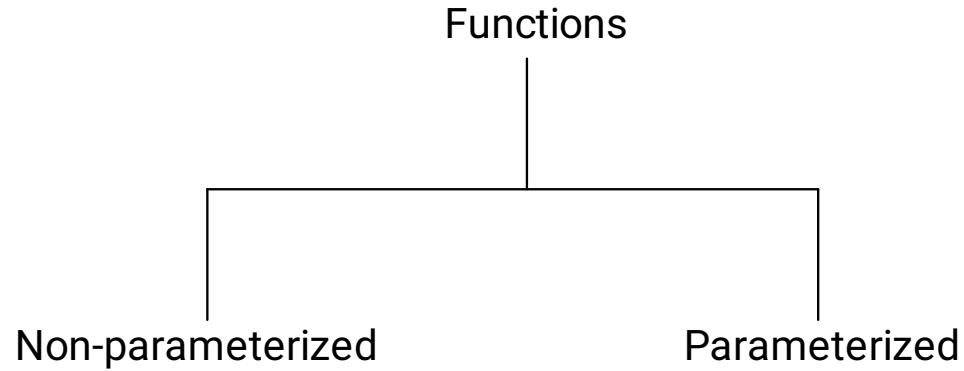
void main()
{
    number = 10;
    printf("I am in main function. My value is %d\n", number);
    fun1();    //function calling, discussed in next topic
    fun2();    //function calling, discussed in next topic
}

/* This is function 1 */
fun1()
{
    number = 20;
    printf("I am in function fun1. My value is %d", number);
}

/* This is function 1 */
fun2()
{
    printf("\nI am in function fun2. My value is %d", number);
}
```

Functions

- Functions are group of statement that together perform a certain task.



Function consists of three parts :-

1. Function Declaration
2. Function Calling
3. Function Definition

Functions

Function Declaration

```
#include <stdio.h>
```

```
void addition()  
{  
    int a,b,c;  
    printf("\n Enter two numbers: ");  
    scanf("%d%d",&a,&b);  
    c=a+b;  
    printf("\n Addition is: %d",c);  
}
```

1

Function Definition

```
void main()  
{  
    printf("\n *****Addition Function*****");  
    addition();  
}
```

Function Calling

```
#include <stdio.h>
```

```
void addition();
```

```
void main()
```

```
{  
    printf("\n *****Addition Function*****");  
    addition();  
}
```

2

```
void addition()
```

```
{  
    int a,b,c;  
    printf("\n Enter two numbers: ");  
    scanf("%d%d",&a,&b);  
    c=a+b;
```

```
    printf("\n Addition is: %d",c);
```

```
}
```

Parameterized Functions (Passing Arguments)

Actual parameters are passed when calling the function and formal parameters are passed during function definition.

```
#include <stdio.h>

void addition(int x, int y)
{
    int c;
    c=x+y;

    printf("\n Addition is: %d",c);
}

void main()
{
    int a,b,c;
    printf("\n *****Addition Function*****");
    printf("\n Enter two numbers: ");
    scanf("%d%d",&a,&b);
    addition(a,b);
}
```

```
#include <stdio.h>
void addition(int, int);
void main()
{
    int a,b,c;
    printf("\n *****Addition Function*****");
    printf("\n Enter two numbers: ");
    scanf("%d%d",&a,&b);
    addition(a,b);
}

void addition(int x, int y)
{
    int c;
    c=x+y;

    printf("\n Addition is: %d",c);
}
```

Passing Arrays to Function

```
#include <stdio.h>
int array_function(int arr[]);
void main()
{
    int arr[4]={12,3,4,5};
    int d;
    d=add(arr);
    printf("The sum of array elements is: %d",d);
}

int add(int arr[])
{
    int i,c=0;
    for(i=0;i<4;i++)
    {
        c=c+arr[i];
    }
    return c;
}
```

```
#include <stdio.h>
int array_function(int *arr);
void main()
{
    int arr[4]={12,3,4,5};
    int d;
    d=add(arr);
    printf("The sum of array elements is: %d",d);
}

int add(int *arr)
{
    int i,c=0;
    for(i=0;i<4;i++)
    {
        c=c+arr[i];
    }
    return c;
}
```

- In C, when we pass an array to a function say add(), it is always treated as a pointer by add().
- In other words, compiler only passes the starting address of the array. To store address we need pointers.

Recursion

- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function.

Some points to remember:-

1. You must have some base condition to terminate the recursion.
2. If there is no base condition, stack overflow will occur.

```
#include <stdio.h>
int recursive(int i);
void main()
{
    int value=5;
    int recursive_value;
    recursive_value=recursive(value);
    printf("The factorial of %d is %d",value,recursive_value);
}

int recursive(int i)
{
    if(i<=1)
        return 1;
    else
        return i * recursive(i-1);
}
```

Direct Recursion and Indirect Recursion

A function fun is called direct recursive if it calls the same function. A function is called indirect recursive if it calls another function say fun_new() and fun_new() calls fun() directly or indirectly.

```
//An example of direct Recursion
void fun()
{
    //Some code
    fun();
    //Some code;
}
```

```
//An example of Indirect recursion
void fun()
{
    //Some code;
    fun_new();
    //Some code;
}
void fun_new()
{
    //Some code;
    fun();
    //Some code;
}
```

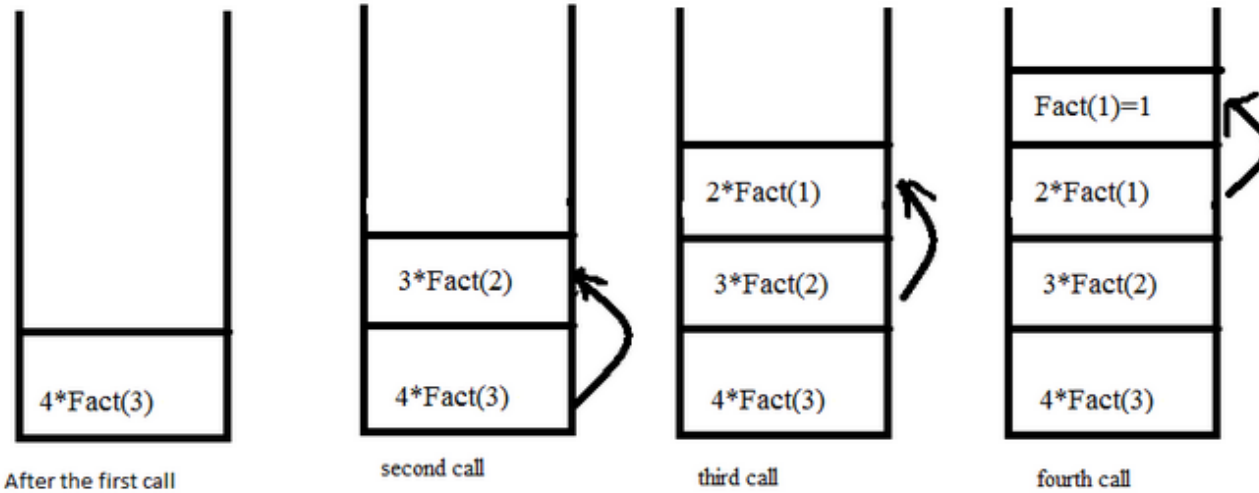
Tail and Non- Tail Recursion

A recursive function is tail recursive when recursive call is the last thing executed by the function. For example the following C++ function print() is tail recursive.

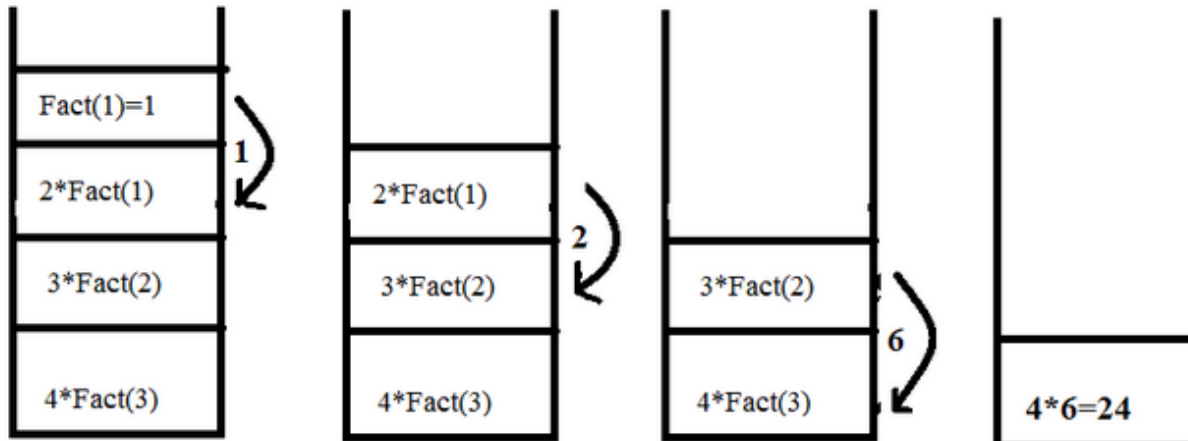
```
// An example of tail recursive function
void print(int n)
{
    if (n < 0) return;
    printf("%d",n);
    // The last executed statement is recursive call
    print(n-1);
}
```

Memory structure for Recursion

When function call happens previous variables gets stored in stack



Returning values from base case to caller function



Structures

A structure is a derived datatype. It is a collection of heterogeneous datatypes. Structure can have different types of variables stored in it like int, float, char string array etc.

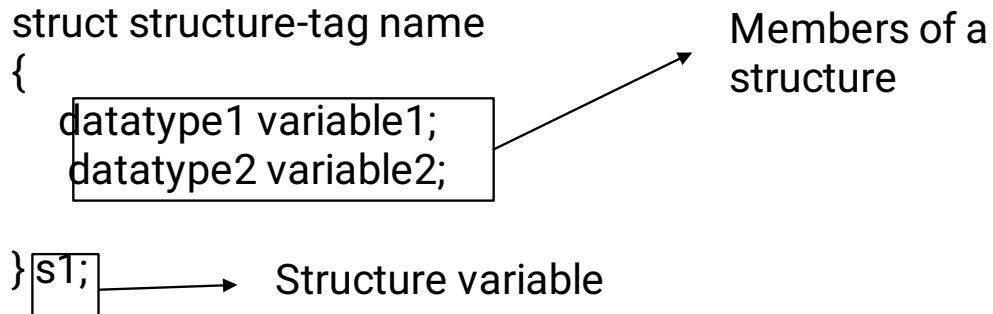
- Disadvantage of an array
Stores similar kind of datatype.

Syntax of Structure:-

```
struct structure-tag name
{
    datatype1 variable1;
    datatype2 variable2;
} s1;
```

Members of a structure

Structure variable



Defining a structure in C

A structure variable can either be declared with structure declaration or as a separate declaration like basic types.

```
struct Student
{
    int roll_no;
    char name[20];
    float percentage;
} s1,s2;
```

1

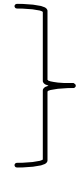
```
struct Student
{
    int roll_no;
    char name[20];
    float percentage;
}
void main()
{
    struct Student s1,s2;
}
```

2

Initializing a Structure

Structure members **cannot be** initialized with declaration. For example the following C program fails in compilation.

```
struct Student
{
    int roll_no=1234;
    char name[20]={“XYZ”};
    float percentage = 71.2;
}
```



Compilation error. Because memory is not allotted at this point.

```
#include <stdio.h>
struct Student
{
    int roll_no;
    char name[20];
};
void main()
{
    struct Student s1={1234,"Amit"};
    printf("\n%s",s1.name);
    s1.roll_no=4321;
    printf("\n%d",s1.roll_no);
    scanf("%s",s1.name);
    printf("\n%s",s1.name);
}
```

Array of Structure

```
#include <stdio.h>
struct Student
{
    int roll_no;
    char name[20];
};
void main()
{
    int i;
    struct Student Student[2]; // Array of Structure
    for(i=0;i<2;i++)
    {
        printf("\nEnter roll number for student %d: ",i+1);
        scanf("%d",&Student[i].roll_no);
        printf("\nEnter name for student %d: ",i+1);
        scanf("%s",&Student[i].name);
    }
}
```

```
printf("\n\n");
for(i=0;i<2;i++)
{
    printf("\nThe roll number for student %d is      %d",
i+1,Student[i].roll_no);
    printf("\nThe name for student %d is %s",i+1,Student[i].
name);
    printf("\n\n");
}
}
```

Nested Structure

```
#include<stdio.h>
struct address
{
    char city[20];
    int pin;
    char phone[14];
};
struct employee
{
    char name[20];
    struct address add;
};
void main ()
{
    struct employee emp;
    printf("Enter employee information?\n");
    scanf("%s %s %d %s",&emp.name,&emp.add.city, &emp.add.pin, &emp.add.phone);
    printf("Printing the employee information....\n");
    printf("name: %s\nCity: %s\nPincode: %d\nPhone: %s",emp.name,emp.add.city,emp.add.pin,
    emp.add.phone);
}
```

Passing Structure to function arguments

```
#include <stdio.h>

struct student
{
    int id;
    char name[20];
    float percentage;
};

void func(struct student recordDetail);
int main()
{
    struct student record;
    printf("\n Enter student details:");
    scanf("%d %s %f",&record.id,&record.name,
&record.percentage);

    func(record);
    return 0;
}
```

```
void func(struct student recordDetail)
{
    printf(" Id is: %d \n", recordDetail.id);
    printf(" Name is: %s \n", recordDetail.name);
    printf(" Percentage is: %f \n", recordDetail.
percentage);
}
```

Returning Structure

```
#include<stdio.h>

struct Employee
{
    int Id;
    char Name[25];
    int Age;
    long Salary;
};

struct Employee Input();      //Statement 1
void main()
{
    struct Employee Emp;

    Emp = Input();
    printf("\n\nEmployee Id : %d",Emp.Id);
    printf("\nEmployee Name : %s",Emp.Name);
    printf("\nEmployee Age : %d",Emp.Age);
    printf("\nEmployee Salary : %ld",Emp.Salary);
}
```

```
struct Employee Input()
{
    struct Employee E;

    printf("\nEnter Employee Id : ");
    scanf("%d",&E.Id);

    printf("\nEnter Employee Name : ");
    scanf("%s",&E.Name);

    printf("\nEnter Employee Age : ");
    scanf("%d",&E.Age);

    printf("\nEnter Employee Salary : ");
    scanf("%ld",&E.Salary);

    return E;      //Statement 2
}
```

Pointers in C

A pointer is a variable that stores the address of another variable.

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int var = 20;
```

```
    int *ip;
```

```
    ip = &var;
```

```
    printf("Address of var variable: %x\n", &var);
```

```
    printf("Address stored in ip variable: %x\n", ip );
```

```
    printf("Value of *ip variable: %d\n", *ip );
```

```
    return 0;
```

```
}
```

Pointer Arithmetic

On pointers we can perform only ++,--,+,- arithmetic operations.

```
#include <stdio.h>

int main ()
{
    int var = 20;
    int *ip;

    ip = &var;

    printf("\nAddress of pointer ip is: %d",ip);
    ip++;
    printf("\nAddress of pointer ip is: %d",ip);
    ip--;
    printf("\nAddress of pointer ip is: %d",ip);
    return 0;
}
```

Address of pointer ip is: 6356728

Address of pointer ip is: 6356732

Address of pointer ip is: 6356728

```
#include <stdio.h>

int main ()
{

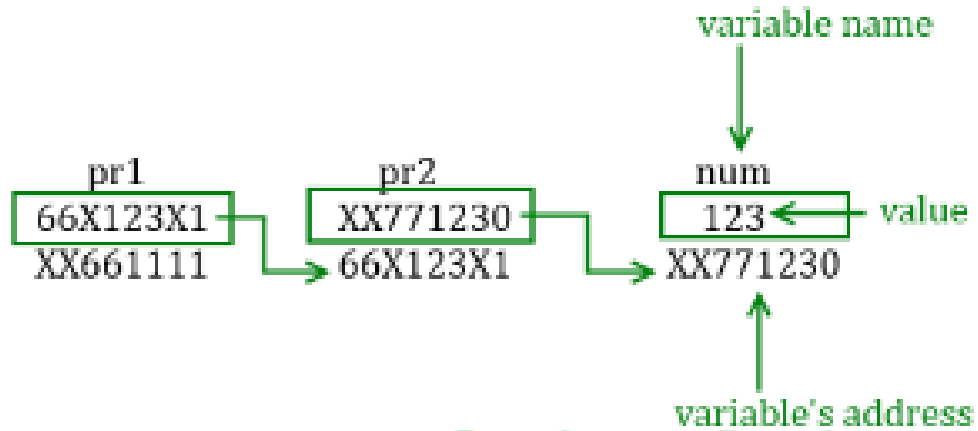
    char ch='a';
    char* cp = &ch;

    printf("\nAddress of pointer cp is: %d",cp);
    cp++;
    printf("\nAddress of pointer cp is: %d",(cp+1));
    cp--;
    printf("\nValue of pointer cp is: %d",*(cp+1));
    return 0;
}
```

Address of pointer cp is: 6356723
Address of pointer cp is: 6356725
Value of pointer cp is: 20

Pointer to Pointer

A pointer will store the address of variable. Another pointer will store the address of first pointer. This is also called as double pointer.



```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int num =123;
```

```
    int *pr2=&num;
```

```
    int **pr1=&pr2;
```

```
    printf("\n Value of variable: %d",num);
```

```
    printf("\n Value of variable: %d",*pr2);
```

```
    printf("\n Value of variable: %d",**pr1);
```

```
    printf("\n Address of variable: %d",&num);
```

```
    printf("\n Address of pointer2: %d",pr2);
```

```
    printf("\n Address of pointer1: %d",pr1);
```

```
    **pr1=321;
```

```
    printf("\n Value of variable: %d",*pr2);
```

```
}
```

Pointer as Function Argument or Call by Reference

```
#include <stdio.h>
void swap(int *x, int *y);
int main ()
{
    int a,b;
    scanf("%d %d",&a,&b);
    printf("\nValues before swapping: %d %d",a,
b);
    swap(&a,&b);
    printf("\nValues after swapping: %d %d",a,b);
}
void swap(int *x, int *y)
{
    int temp;
    temp=*x;
    *x=*y;
    *y=temp;
}
```

Pointer to an Array

